

Циклы в SCSS

Что такое циклы в SCSS?

Циклы в SCSS - это конструкции, которые позволяют повторять блоки кода несколько раз. Они особенно полезны для создания повторяющихся стилей, генерации классов и автоматизации рутинных задач. SCSS предлагает несколько типов циклов, каждый из которых имеет свои особенности и применение.

Типы циклов в SCSS

Цикл @for

Цикл `@for` используется для повторения блока кода определенное количество раз. Он имеет две формы:

```
// От начала до конца включительно (through)
@for $i from 1 through 5 {
  .item-#{$i} {
    width: 20px * $i;
  }
}

// От начала до конца не включительно (to)
@for $i from 1 to 5 {
  .col-#{$i} {
    width: 20% * $i;
  }
}
```

Результат компиляции:

```
/* through (включительно) */
.item-1 {
  width: 20px;
}
.item-2 {
  width: 40px;
}
.item-3 {
  width: 60px;
}
.item-4 {
  width: 80px;
}
.item-5 {
```

```

    width: 100px;
}

/* to (не включительно) */
.col-1 {
    width: 20%;
}
.col-2 {
    width: 40%;
}
.col-3 {
    width: 60%;
}
.col-4 {
    width: 80%;
}

```

Цикл @each

Цикл `@each` используется для перебора элементов в списке или карте (map):

```

// Перебор списка
$colors: red, green, blue, yellow;

@each $color in $colors {
    .text-#{ $color } {
        color: $color;
    }
}

// Перебор списка с индексом
@each $color in $colors {
    $i: index($colors, $color);
    .bg-#{ $color } {
        background-color: $color;
        z-index: $i;
    }
}

// Перебор карты (map)
$font-sizes: (
    small: 12px,
    medium: 16px,
    large: 24px
);

@each $name, $size in $font-sizes {
    .text-#{ $name } {
        font-size: $size;
    }
}

```

```

}

// Деструктуризация нескольких значений
$buttons: (
  primary: (blue, white),
  secondary: (gray, black),
  danger: (red, white)
);

@each $name, $colors in $buttons {
  $bg: nth($colors, 1);
  $text: nth($colors, 2);

  .btn-#{$name} {
    background-color: $bg;
    color: $text;
  }
}

```

Цикл @while

Цикл `@while` выполняется, пока указанное условие истинно:

```

$i: 1;

@while $i <= 5 {
  .item-#{$i} {
    width: 10px * $i;
  }
  $i: $i + 1;
}

```

Практические примеры использования циклов

Создание системы сетки

```

$columns: 12;

@for $i from 1 through $columns {
  .col-#{$i} {
    width: percentage($i / $columns);
  }
}

```

Создание классов для отступов

```

$spacer: 4px;
$steps: 10;

@for $i from 1 through $steps {
  // Margin
  .m-#{ $i } {
    margin: $spacer * $i;
  }

  // Padding
  .p-#{ $i } {
    padding: $spacer * $i;
  }

  // Margin по направлениям
  .mt-#{ $i } { margin-top: $spacer * $i; }
  .mr-#{ $i } { margin-right: $spacer * $i; }
  .mb-#{ $i } { margin-bottom: $spacer * $i; }
  .ml-#{ $i } { margin-left: $spacer * $i; }

  // Padding по направлениям
  .pt-#{ $i } { padding-top: $spacer * $i; }
  .pr-#{ $i } { padding-right: $spacer * $i; }
  .pb-#{ $i } { padding-bottom: $spacer * $i; }
  .pl-#{ $i } { padding-left: $spacer * $i; }
}

```

Создание цветовой палитры

```

$base-color: #3498db;
$steps: 5;

@for $i from 1 through $steps {
  .bg-lighten-#{ $i } {
    background-color: lighten($base-color, $i * 10%);
  }

  .bg-darken-#{ $i } {
    background-color: darken($base-color, $i * 10%);
  }
}

```

Создание анимаций с задержкой

```

$items: 5;

@for $i from 1 through $items {

```

```

.fade-in-#{ $i } {
  animation: fadeIn 1s ease-in-out;
  animation-delay: 0.2s * $i;
}
}

@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

```

Генерация классов из карты (map)

```

$breakpoints: (
  xs: 576px,
  sm: 768px,
  md: 992px,
  lg: 1200px,
  xl: 1400px
);

@each $name, $width in $breakpoints {
  @media (min-width: $width) {
    .container-#{ $name } {
      max-width: $width;
    }
  }
}

```

Создание утилитарных классов для текста

```

$text-alignments: left, center, right, justify;

@each $align in $text-alignments {
  .text-#{ $align } {
    text-align: $align;
  }
}

$font-weights: (
  light: 300,
  regular: 400,
  medium: 500,
  bold: 700,
  black: 900
);

@each $name, $weight in $font-weights {

```

```
.font-#{ $name } {  
    font-weight: $weight;  
}  
}
```

Вложенные циклы

Вы можете комбинировать циклы для создания более сложных структур:

```
$rows: 3;  
$cols: 3;  
  
@for $row from 1 through $rows {  
    @for $col from 1 through $cols {  
        .grid-#{ $row }-#{ $col } {  
            grid-row: $row;  
            grid-column: $col;  
        }  
    }  
}
```

Комбинирование циклов с условиями

```
$columns: 12;  
  
@for $i from 1 through $columns {  
    .col-#{ $i } {  
        width: percentage($i / $columns);  
  
        @if $i < 4 {  
            // Маленькие колонки  
            font-size: 12px;  
        } @else if $i < 8 {  
            // Средние колонки  
            font-size: 14px;  
        } @else {  
            // Большие колонки  
            font-size: 16px;  
        }  
    }  
}
```

Лучшие практики использования циклов

1. Выбирайте подходящий тип цикла

- `@for` - когда нужно повторить код определенное количество раз
- `@each` - для перебора элементов списка или карты
- `@while` - для более сложной логики с условиями

2. Используйте интерполяцию для генерации имен селекторов

- `#{$variable}` позволяет вставлять значения переменных в имена селекторов

3. Избегайте слишком сложных вложенных циклов

- Большое количество вложенных циклов может привести к сложному для понимания коду
- Старайтесь ограничиваться 2-3 уровнями вложенности

4. Используйте переменные для параметров циклов

- Определяйте количество итераций и другие параметры через переменные
- Это упрощает поддержку и изменение кода в будущем

5. Комментируйте сложные циклы

- Добавляйте комментарии, объясняющие логику работы сложных циклов

6. Следите за производительностью

- Избегайте генерации слишком большого количества CSS-правил
- Помните, что весь сгенерированный CSS будет загружен в браузер

Заключение

Циклы в SCSS - это мощный инструмент для автоматизации создания стилей. Они позволяют значительно сократить объем кода, сделать его более поддерживаемым и гибким. Правильное использование циклов особенно полезно при создании систем сеток, утилитарных классов, цветовых палитр и других повторяющихся элементов дизайн-системы. Понимание различных типов циклов и их особенностей поможет вам выбрать наиболее подходящий инструмент для каждой конкретной задачи.